

Práctica 1: variables compartidas

HOJA Nº 1

FECHA

Operación atómica: ningún otro proceso puede modificar o acceder a op. relacionadas.

Sentencia atómica = sentencia de máquina.

Historias = órdenes en las q se intercalan las sentencias

Soluciones de grano grueso / sentencia await

<await (B)> sentencias

El proceso se detiene en este pto. hasta q la cond. B es verdadera y en ese momento y atómicamente ejecuta las sentencias

La atomicidad empieza desde el momento q se está haciendo el último chequeo de B

Brinda sincronización x exclusión mutua y/o x cond. $\rightarrow$ xq no se ejecuta hasta B=true

las sentencias dentro se ejecutan atómicamente

Await p/ exclusión mutua <sentencias>

Se ejecutan atómicamente, sólo un proceso a la vez ejecuta esa sección crítica.

Cualquier proceso esperando la SC entra, no hay un orden definido, no FIFO, LIFO, etc.

Await sólo p/ sincronizar x cond. <await B> detiene al proceso en el punto.

El proceso único se detiene hasta q B es verdadero.

La idea es usar la > cant. de variables locales evitando <> ya q reduce la concurrencia

1) Suponga q todas las variables están inicializadas en 0 antes de empezar. Indique cuáles son válidas

a) En algún caso al terminar, x es 56

b) " 22

c) " 23

1) lo importante aquí son las historias no el orden de ejecución

P1::

if (x=0) then
y = 4*2;
x := y + 2;

P2::

if (x > 0) then
x := x + 1;

P3::

x := (x*3) + (x*2) + 1

P1::

- 1) Load PosMemX
- 2) Store 4, Acumulador
- 3) Mul 2, acumulador
- 4) Store Acumulador, PosMemY
- 5) Store PosMemY, acumulador
- 6) Add 2, acumulador
- 7) Store acumulador, PosMemX

P2::

- 8) Load PosMemX
- 9) Add 1, PosMemX

P3::

- 10) Load PosMemX
- 11) Add 1, acumulador

Posibles historias

1, 2, 3, 4, 5, 6, 7, 8

NOTA

NOTA

2) Realice una solux concurrente de grano grueso (usando $\langle \rangle$ y/o $\langle \text{await } B; S \rangle$) p/ el sig. problema. Dado un nro. N verifique cuantas veces aparece ese nro en un arreglo de longitud M . Escriba las pre-condiciones necesarias.

```
int cantidad = 0;
int arreglo[M];
```

```
process calcular [i = 1 to P] {
```

```
  int inicio, fin;
```

```
  int temp;
```

```
  int tamaño = M/P;
```

```
  inicio = (i-1) * tamaño;
```

```
  fin = (inicio + tamaño - 1);
```

```
  for j = inicio to fin si arreglo[j] = N then temp := temp + 1 af
  cantidad := cantidad + temp
```

3) Dada la sig. solux de grano grueso.

2) Indicar si el sig. código funciona p/ resolver el problema de Productor/consumidor d un buffer de tamaño N . Si no, hacer las modificaciones necesarias.

```
int cant = 0; int pri_occupada = 0; int pri_vacia = 0; int buffer[N];
```

```
Process Productor::
```

```
{ while (true)
```

```
  { produce elemento
```

```
    <await cant < N; cant++>
```

```
    buffer[pri_vacia] = elemento;
```

```
    pri_vacia = (pri_vacia + 1) mod N;
```

```
  }
```

```
}
```

```
Proceso Consumidor::
```

```
{ while (true)
```

```
  { <await (cant > 0); cant-->
```

```
    elemento = buffer[pri_occupada];
```

```
    pri_occupada = (pri_occupada + 1) mod N;
```

```
    consume elemento
```

```
  }
```

Puede pasar de que $\text{cant} = 1$ pero el productor todavía NO guardó el elemento quedando

```
Process Productor::
```

```
{ while (true)
```

```
  // produce elemento
```

```
  <await cant < N; cant++>
```

```
  buffer[pri_vacia] = elemento;
```

```
  <cant++>
```

```
  pri_vacia = (pri_vacia + 1) mod N;
```

```
}
```

```
}
```

```
Process Consumidor::
```

```
{ while (true)
```

```
  { <await (cant > 0); >
```

```
    elemento = buffer[pri_occupada];
```

```
    <cant-->
```

```
    pri_occupada = (pri_occupada + 1) mod N;
```

```
    consume elemento
```

```
}
```

② opción poner todo dentro de $\langle \text{await cant} > N \rangle$ y $\langle \text{await cant} > 0 \rangle$

```
Productor::
```

```
...
```

```
<await cant < N; buffer[pri_vacia] = elemento; cant++>
```

```
...
```

```
Consumidor:: <await cant > 0; elemento = buffer[pri_occupada]; cant-->
```

```
...
```

Respecto a $pr_varia/pr_ocupado$ y $\text{mod } N$, el buffer se va llenando y vaciando indefinidamente x lo que se trata como un buffer circular, al llegar a la posición N vuelve al inicio p/ seguir el muestreo.
 $\text{cant} < N$ p/ validar de q' haya lugar. Cuando llega en el índice $> N-1$, vuelve a la pos $= 0$. $(pr_varia + 1) \text{ mod } N$ da 0, 1, 2, etc.

b) Modificar p/ q' funcione c/ C Productores y N consumidores
 Aquí lo importante es poner como SC pr_varia y $pr_ocupado$ ya q' ahora será accedido x más de un proceso

int $\text{cant} = 0$; int $pr_ocupado = 0$ int $pr_varia = 0$ int $\text{buffer}[N]$;

Process $\text{Productor } [i = 1..C]$

{ while (true)

// produce elemento

< await $\text{cant} < N$; $\text{buffer}[pr_varia] = \text{elemento}$;

$\text{cant}++$;

$pr_varia = (pr_varia + 1) \text{ mod } N$; >

}

}

Process $\text{Consumidor } [j = 1..N]$

{ while (true)

// await $\text{cant} > 0$;

$\text{elemento} = \text{buffer}[pr_ocupado]$;

$\text{cant}--$

$pr_ocupado = (pr_ocupado + 1) \text{ mod } N$; >

consume elemento

}

}

4) Resolver con sentencias **AWAIT** (<> y < await $B; S$ >. Un sistema operativo mantiene 5 instancias de recursos almacenados en una cola, cuando un proceso necesita usar una instancia del recurso le saca de la cola y la vuelve a depositar cuando termina
 (recurso, índice inicio en 0 0.1, sección 4)

recurso $\text{cola}[5]$; int $\text{disponible} = 5$

Process $\text{Proceso } [i = 1..N]$ {

< await $\text{disponible} > 0$;

recurso = $\text{cola.pop}()$;

$\text{disponible}--$; >

usar recurso;

< $\text{cola.push}(\text{recurso})$;

$\text{disponible}++$; >

}

① ¿recurso necesita alguna condición p/ flag de cuando un proceso necesita un recurso?

5) En cada item debe realizar una solux concurrente de grano grueso (usando $\langle \rangle$ y/o $\text{await } B, S \rangle$ p/ el sig. problema, teniendo en cuenta las condiciones indicadas. Hay N personas g_i deben imprimir un trabajo d_i .

a) Implemente una solux suponiendo g_i hay una única impresora compartida y todas las personas y las mismas l_i deben usar de a una persona a la vez, sin importar el orden. Existe una f_x Imprimir (documento) llamada x la persona g_i simula el uso de la impresora. Solo se deben usar x los procesos g_i representan a las personas.

```
bool disponible = true;
Process Persona N [i=1..J] {
  < await disponible = true; disponible = false; >
  Imprimir (documento);
  < disponible = true >
}
```

b) Modifique la solux p/ g_i se respete un orden de llegada.

```
bool disponible = true; cola personas;
Process Persona [i=0..N] {
  cola.push(i);
  < await disponible = true AND cola[0] = i;
  disponible = false;
  cola.pop(); >
  Imprimir (documento);
  < disponible = true > i;
}
```

```
Process Persona [i=0..N] {
  if disponible = true => < disponible = false; >
  Imprimir (documento);
  < disponible = true >
  else {
    < cola.push(i); >
    < await disponible = true AND proximo == i;
    disponible = false; >
    Imprimir (documento);
    < cola.pop(); >
    proximo = i;
  }
  < disponible = true > proceso
```

c) Modifique la solux de a) p/ el caso en que se deba respetar el orden dado por el identificador del proceso (cuando está libre la impresora, de los procesos que han solicitado su uso la debe usar el g_i tenga menor identificador)

```
bool disponible = true; cola personas;
Process Persona [i=1..J] {
  cola.push(i);
  < await disponible = true AND cola.min() == i;
  disponible = false;
  cola.get_Minimo(); >
  Imprimir (documento);
  < disponible = true > i;
}
```

Si llega y está disponible
NO pusheo, paso a usarla.

En cola.push/cola.pop()
ambas son SC, en un pop
podría tomarse algo g_i
arajo de pushearse.

cola[0] no existe, otra
manera p/ tomar el índice
del pop.

solución ok 5) b) atrás.

d) Modifique la solución de b) para el caso en q' además hay un proceso Coordinador q' le indica a cada persona q' es su turno de usar la impresora.

// Debe encontrar la manera de cómo avisarle a la persona cuándo usar la impresora y viceversa está avisándole al coordinador q' ya terminó de usarla

personas cola; int turno = -1;

bool imprimiendo = false;

Process Persona [i = 1..N] {

cola.push(i);

<await turno == i>

Imprimir(documento);

imprimiendo = false;

}

Process Coordinador {

while (true) {

-- if ! cola.empty() == false AND turno == -1;

turno = cola.pop();

imprimiendo = true;

<await imprimiendo == false>

turno = -1;

b) Dada la sig. solux p/ el problema de la SC entre dos procesos (suponiendo q' tanto SC como SNC son segmentos de código no finitos, es decir q' terminan en algún momento, indicar si cumple c/ las cond. requeridas.

int turno = 1;

Process SC1::

{ while (true) {

while (turno == 2) skip;

SC

turno = 2;

SNC;

}

}

Process SC2::

{ while (true) {

if (turno == 1) skip;

SC

turno = 1;

SNC;

}

4 condiciones

① Exclusión mutua: a lo sumo un proceso está en su SC

② Ausencia de deadlock: si 2 o + procesos tratan de entrar a una SC solo uno tendrá éxito.

③ Ausencia de demora innecesaria: un proceso trata de entrar a una SC y los otros están en una SNC

④ Eventual entrada: eventual' entrarán a una SC

NO se cumple ③ ausencia de demora innecesaria xq' sea SC2, sigue tomando el turno bloqueando/haciendo esperar a SC1 a pesar de estar en una SNC